Task-11:-Requirement Analysis

1. Introduction

1.1 Purpose The purpose of this document is to outline the functional and technical requirements for deploying the Bookstore API on AWS using a 2-tier architecture. The intended audience includes the DevOps Engineering team, Lead Architects, and Quality Assurance testers at ABC Technologies.

1.2 Scope

- **In-Scope:** Provisioning AWS EC2 for application hosting, setting up MongoDB (Atlas or EC2), configuring networking/security groups, and verifying API-DB integration.
- **Out-of-Scope:** Frontend development, SSL/TLS certificate implementation (HTTPS), and CI/CD pipeline automation (unless specifically requested in future sprints).

1.3 Definitions, Acronyms, and Abbreviations

- **AWS:** Amazon Web Services
- **EC2:** Elastic Compute Cloud
- **FastAPI:** A modern, high-performance web framework for building APIs with Python.
- **SGR:** Security Group Rule
- **NoSQL:** Non-Relational Database (MongoDB)

1.4 References

- Bookstore API Repository: [GitHub - pushkar-iamops/bookstore-api]
- FastAPI Documentation: FastAPI
- AWS Shared Responsibility Model.

1.5 Overview This document transitions from project objectives to specific functional use cases, followed by security/performance requirements, risk analysis, and final sign-off.

---

2. Project Overview

2.1 Project Objectives

- Achieve a logical separation between the web application and the database.
- Ensure the Bookstore API is reliably accessible via public IP for internal testing.
- Validate the database connection using environment variables.

2.2 Business Goals

- Modernize internal dev/test environments by moving from local hosting to the cloud.
- Demonstrate architectural best practices (2-tier) to minimize the blast radius of potential failures.

2.3 Background ABC Technologies currently manages the Bookstore API locally. To scale testing and allow collaborative development, they require a cloud-native deployment on AWS that mimics production-like security standards.

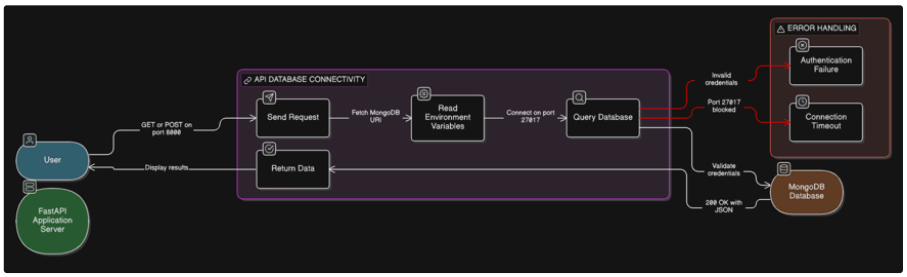
2.4 Constraints

- **Time:** Must be completed within a 3-hour window.
- **Budget:** Use AWS Free Tier eligible resources (t2.micro/t3.micro) where possible.
- **Database:** Must use MongoDB (NoSQL).

---

3. Functional Requirements

3.1 Use Case Diagram



3.2 Use Cases

3.2.1 Use Case 1: API-Database Connectivity

- **Description:** The FastAPI application establishes a connection to the MongoDB instance to fetch or store book records.
- **Actors:** FastAPI Application Server, MongoDB Database.
- **Preconditions:** EC2 instance is running; MongoDB is active and reachable via port 27017.
- **Postconditions:** API returns a `200 OK` status and valid JSON data from the DB.
- **Flow of Events:**
  a. User sends a GET/POST request to the EC2 Public IP on port 8000.
  b. Application reads `MONGODB_URI` from the environment variables.
  c. Application sends a query to the MongoDB IP/DNS.
  d. Database validates credentials and returns data.
- **Alternative Flows:**
  a. **Connection Timeout:** If port 27017 is blocked by SGR, the application logs a connection error.
  b. **Authentication Failure:** If the URI contains the wrong password, the DB denies access.

---

4. Non-Functional Requirements

4.1 Performance Requirements

- The API should respond to basic CRUD requests in under 500ms.
- The system should support at least 5 concurrent users for internal testing.

4.2 Security Requirements

- **Data Protection:** Database credentials must be stored in environment variables, never hardcoded.
- **Network Security:** The Database should only accept traffic from the Application Server's IP address.

4.3 Usability Requirements

- FastAPI `/docs` (Swagger UI) must be enabled to allow testers to interact with the API without writing custom scripts.

4.4 Reliability Requirements

- The application process should be managed by a process manager (e.g., `systemd`) to ensure it restarts upon instance reboot.

4.5 Compliance Requirements

- All resources must be tagged with `Project: Bookstore-API` and `Owner: Vraj-Lalwala` for internal tracking.

---

5. Assumptions and Dependencies

5.1 Assumptions

- The GitHub repository is public and accessible from the EC2 instance.

- The user has sufficient AWS IAM permissions to launch EC2 and modify Security Groups.

5.2 Dependencies

- **Python/Pip:** The application cannot run without the correct runtime environment.
- **MongoDB Atlas/Server:** The API functionality depends entirely on the availability of the database layer.

---

6. Risk Analysis

6.1 Risk Identification

- **Risk A:** Exposing port 27017 to the public internet (0.0.0.0/0).
- **Risk B:** EC2 instance hitting CPU/Memory limits on Free Tier.

6.2 Risk Mitigation

- **Mitigation A:** Restrict SGR inbound rules to only the Application Server's Security Group ID.
- **Mitigation B:** Monitor resource usage using CloudWatch metrics.

---

7. Glossary

- **Tier:** A functional layer in a software architecture.
- **CRUD:** Create, Read, Update, Delete.
- **URI:** Uniform Resource Identifier (connection string).

---

8. Appendices

- Deployment logs and screenshot of the working `/docs` page will be attached to the final Confluence page.

---

9. Customer Acknowledgement

Name: _____

Title: Senior DevOps Engineer / Stakeholder

Signature: _____

Date: _____